Invoice Agent

Prepared by: Gopal Gorain
Project: Invoice Agent
Repository: souravhub/invoice-agent
Date: 05 October 2026

Problem Statement

Businesses receive invoices in many different formats and often rely on manual data entry to extract and verify information such as vendor details, invoice numbers, dates, line items, taxes, and total amounts. This process is time-consuming, costly, and prone to human error.

The challenge is to build an intelligent invoice-processing system that can automatically extract structured data from invoices, validate financial calculations, detect missing or incorrect information, and safely route uncertain invoices for human review instead of incorrectly approving them.

The system should ultimately help businesses **reduce manual work, improve accuracy, speed up invoice processing, and minimize financial errors.**

Invoice Agent – Project Summary

The Invoice Agent is an AI-powered invoice processing system designed to automate the process of reading, extracting, validating, and reviewing invoice data.

The system accepts invoice PDF files and extracts important information such as vendor name, invoice number, invoice date, purchase order number, currency, line items, subtotal, discount, shipping charges, tax, and total amount.

The AI model is mainly responsible for understanding the invoice and converting unstructured invoice text into structured data. After extraction, Python-based validation rules check the financial values and verify whether the invoice information is consistent.

The overall workflow of the project is:

Invoice PDF → Text Extraction → AI Data Extraction → Data Validation → Error and Calculation Checks → Auto Approval or Human Review

If the invoice passes all validation checks, the system can mark it as auto-approved. If there is missing information, incorrect calculations, ambiguous data, or any other issue, the invoice is sent for human review.

One of the key strengths of this project is that it does not fully depend on AI for financial decisions. AI is used for data extraction, while important calculations and validations are handled using deterministic Python logic. This helps reduce the risk of incorrect invoice approvals.

The project also includes retry logic, error handling, evaluation metrics, and mechanisms to identify potentially unsafe or incorrect results.

The main goal of the Invoice Agent is to reduce manual invoice processing, improve accuracy, save time, and make invoice handling safer and more efficient for businesses.

In the future, the project can be enhanced with features such as OCR for scanned invoices, duplicate invoice detection, purchase order matching, approval workflows, dashboards, APIs, ERP integrations, audit logs, and multi-user support.

Overall, the Invoice Agent can be developed into a complete AI-powered Accounts Payable automation solution for businesses.

Things Completed in the Invoice Agent Project

1. Invoice PDF Processing

The system can accept an invoice PDF and extract its text for further processing.

2. AI-Based Data Extraction

The AI can extract important invoice information such as:

- Vendor name
- Invoice number
- Invoice date
- Purchase Order number
- Currency
- Line items
- Subtotal
- Discount
- Shipping charges
- Tax
- Total amount

3. Structured Output

A predefined schema has been created using Pydantic so that the AI returns invoice information in a fixed and structured format instead of free-form text.

4. Invoice Validation

Python-based validation logic has been implemented to verify extracted invoice information.

The system checks whether important fields are missing and whether financial calculations are consistent.

5. Mathematical Validation

The system verifies invoice calculations using normal Python logic instead of depending on the AI for mathematical decisions.

This helps identify problems such as incorrect subtotal, tax, discount, shipping, or total calculations.

6. Auto Approval and Human Review

The system currently provides two main results:

Auto Approved – The invoice passes the required checks.

Needs Review – The invoice contains missing, uncertain, ambiguous, or inconsistent information and should be checked by a human.

7. Retry Mechanism

A retry mechanism has been implemented for extraction-related problems.

If important information is missing, the AI can retry extraction up to two additional times.

Mathematical errors are not sent back to the AI to be corrected automatically.

8. Ambiguous Date Detection

The system supports different client date formats:

- DMY
- MDY
- Unknown

If the date format is unknown and an invoice contains an ambiguous date such as `04/05/2026`, the system can send the invoice for human review.

9. Multiple AI Provider Support

The project supports both:

- OpenAI
- Google Gemini

The AI provider and model can be changed through environment configuration without changing the application code.

10. Test Invoice Dataset

A test dataset has been created containing **15 synthetic invoice PDFs**.

Answer keys are also available so the extracted values can be compared with the expected values.

11. Automated Evaluation System

An evaluation script has been implemented to automatically test invoices and measure the performance of the Invoice Agent.

The evaluation includes metrics such as:

- Field accuracy
- Silent errors
- Caught errors
- Clean passes
- Over-cautious decisions
- Escalation rate

12. Silent Error Detection

The project specifically measures silent errors.

A silent error occurs when the system extracts incorrect information but still automatically approves the invoice.

Reducing silent errors is one of the major safety objectives of the project.

13. Improvements Already Applied

Several problems identified during testing have already been addressed, including:

- Better handling of discount and shipping values
- Prevention of the AI inventing missing subtotal or tax values

- Improved retry behaviour
- Ambiguous date detection
- Improved escalation rules

14. Evaluation Reports

Evaluation reports are saved in the project so that results before and after improvements can be compared.

The available reports include baseline testing and testing performed after several fixes.

15. Result Storage

When an invoice is processed, the extracted data and final decision can be saved as JSON output for further analysis.

Current Overall Status

The basic Invoice Agent workflow has been successfully implemented:

Invoice PDF → Text Extraction → AI Data Extraction → Validation → Error Checking → Auto Approval / Human Review

The project currently works as a functional Proof of Concept for AI-based invoice extraction and validation.

Improvements Needed in Invoice Agent Project

The current Invoice Agent works well as a Proof of Concept, but several improvements are required before it can be considered a production-ready product.

1. Add OCR Support

Currently, the project mainly works with PDFs that already contain readable text.

It should support:

- Scanned invoices
- Image-based PDFs
- Mobile camera photos
- Low-quality invoices
- Rotated invoices

OCR tools such as Tesseract, Google Document AI, AWS Textract, or Azure Document Intelligence can be integrated.

---

2. Improve Invoice Testing

The current test dataset is still small.

More test invoices should be added with different scenarios such as:

- Different invoice layouts
- Multiple currencies
- Multiple languages
- Missing fields
- Incorrect totals
- Large invoices
- Multi-page invoices
- Credit notes
- Duplicate invoices
- Scanned invoices

A larger real-world dataset will make the evaluation more reliable.

---

3. Add Duplicate Invoice Detection

The system should detect if the same invoice is uploaded more than once.

Example:

Vendor: ABC Ltd
Invoice No: INV-1001

If the same invoice is uploaded again, the system should flag it.

Possible checks:

- Vendor + Invoice Number
- Invoice hash
- Date + Total + Vendor
- Similar invoice detection

---

4. Add Purchase Order Matching

The system currently extracts PO numbers, but stronger validation should be added.

It should support:

2-Way Matching

Invoice ↔ Purchase Order

and later:

3-Way Matching

Invoice ↔ Purchase Order ↔ Goods Receipt

This helps detect incorrect quantity, price, or vendor information.

---

5. Improve Financial Calculations

Financial values should preferably use Python `Decimal` instead of `float`.

Fields such as:

- Subtotal
- Tax
- Discount
- Shipping
- Unit price
- Total

should use precise decimal calculations to avoid floating-point rounding problems.

---

6. Add Human Review Dashboard

Currently, the system can return:

`needs_review`

But a real product should provide a dashboard where a user can see:

- Original invoice
- Extracted data
- Validation errors
- Reason for review
- Approve button
- Reject button

- Edit extracted fields
- Reviewer comments

---

7. Add API Support

Currently the project mainly works through Python scripts.

A REST API should be created using something like FastAPI.

Example:

```
POST /invoices
```

The API should return structured invoice data and status.

This will make the system easier to integrate with web applications and ERP systems.

---

8. Improve Error Handling

The system should have clearer error handling for:

- Invalid PDFs
- Empty PDFs
- Password-protected PDFs
- Corrupted PDFs
- LLM API failures
- Rate limits
- Network errors
- Unsupported file types

Errors should be logged properly instead of crashing the application.

---

9. Add Logging and Audit Trail

Every invoice action should be recorded.

For example:

Invoice uploaded → Data extracted → Validation performed → Sent for review → Approved by user

This is especially important for financial applications.

---

10. Improve Security

Before using the system with real customer invoices, security should be improved.

Important areas include:

- API authentication
- Role-based access
- Encryption
- Secure API key storage
- Data deletion policy
- Invoice retention policy
- Audit logs
- Sensitive-data protection

---

11. Add Database Support

A production system needs a database.

It should store:

- Invoice details
- Processing status
- Vendor information
- Validation errors
- Review history
- Duplicate detection information
- User activity

Possible databases:

- PostgreSQL
- MySQL
- MongoDB

---

12. Improve Package Structure

The project should be converted into a proper Python package.

Recommended additions:

- `pyproject.toml`
- `src/invoice_agent/`
- Unit tests
- Integration tests
- Versioning
- CLI commands
- Build configuration

Then it could be installed as:

```
pip install invoice-agent
```

---

13. Add Docker Support

Docker should be added so the project can run consistently on different servers and developer machines.

Example deployment:

```
Docker → API → Invoice Agent → Database → LLM
```

---

14. Add CI/CD

GitHub Actions can be added to automatically run:

- Unit tests
- Integration tests
- Code quality checks
- Package builds
- Deployment checks

This improves development quality and reduces deployment errors.

---

15. Add Multiple Invoice Detection

Sometimes one PDF may contain multiple invoices.

The system should detect and split them instead of treating the full PDF as one invoice.

---

16. Improve Document Type Detection

The system should distinguish between:

- Invoice
- Credit Note
- Debit Note
- Purchase Order
- Receipt
- Proforma Invoice

Currently, these documents may be processed as if they were normal invoices.

---

17. Add Vendor Validation

The system can be improved by maintaining vendor records and validating extracted vendor information.

Checks could include:

- Vendor name
- Vendor ID
- Bank account
- GST/VAT number
- Currency
- Payment terms

---

18. Add India-Specific Invoice Support

For the Indian market, the product could support:

- GSTIN
- CGST
- SGST
- IGST
- HSN/SAC
- Place of Supply
- Taxable Amount
- GST validation

This can make the product more useful for Indian businesses.

---

19. Improve Monitoring

Production monitoring should track:

- Number of invoices processed
- Auto-approved invoices
- Human-review invoices
- Extraction failures
- Silent errors
- Average processing time
- LLM API cost
- Token usage

---

20. Add ERP and Accounting Integrations

For commercial use, integrations should be added with platforms such as:

- SAP
- Oracle
- Microsoft Dynamics
- QuickBooks
- Xero
- Zoho Books
- Tally

This will allow invoice data to automatically move into a customer's accounting system.